

Recall or Relearn? A Mixture of In-Context Experts for Recurring Concept Drift

Anonymous Authors

Abstract—Tabular foundation models (TFMs) learn a data stream through their context, so a past concept can be recalled by putting its rows back into the context, at no training cost. Current in-context stream learners do not use this: a sliding context forgets a concept once it leaves the window, and a detector that resets the context is unsafe, losing up to 9.6 points to a plain window on real streams. We ask when recalling a concept is worth more than relearning it, and how to recall without paying when there is nothing to recall. We show that the value of memory is set by the TFM’s learning curve: the gain of recall over relearning is the area between the learning curve and the accuracy of the stored context. We propose MICE, a mixture of in-context experts, fast windows and stored contexts, under a meta expert with two time scales. The fast level identifies a recurring concept within one batch when the experts are separated by a margin; the slow level has a discounted regret of at most $\ln K/\eta$ against the plain window at any time, with no assumption on the stream. On a pre-registered grid of recurring concepts, the gain predicted from learning curves measured beforehand ranks the observed gains with Spearman 0.94, and MICE exceeds a reset detector on every stream with hard concepts, by 8.5 points on the hardest cell, and a sliding window by 24 points on average. On 29 real streams MICE is more accurate than all four in-context baselines on 23 and never more than 0.25 points below the plain window; a pre-registered test on six held-out segments confirms both.

Index Terms—Concept drift, recurring concepts, tabular foundation models, in-context learning, data streams, online learning, mixture of experts.



1 Introduction

A data stream revisits its past: seasons, weekdays, market regimes and sensor modes return. A learner that kept what it knew about a concept can use it again when the concept recurs [1, 2]. For a trained model this means keeping a pool of models and deciding which one to reuse [3–5]. Tabular foundation models (TFMs) such as TabPFN [6] and TabICL [7] change the cost of this idea. A TFM approximates posterior prediction in one forward pass given a context of labelled rows [8, 9]; its knowledge of a concept is a set of rows, and recalling the concept means conditioning on them. Memory needs no training.

TFMs have already replaced incremental trees as stream learners: with a sliding memory they outperform adaptive tree ensembles on non-stationary benchmarks [10], and a prior with drifting causal models improves them under temporal shift [11]. Neither line uses the free memory. A sliding context forgets a concept once it leaves the window, and a drift-extrapolating prior models a trend rather than a return. Three observations show what an in-context stream learner is missing, and what a remedy has to respect.

Observation 1 (Sliding contexts forget). In the five batches after a concept recurs, a TFM with a FIFO context or with the dual memory of Lourenço et al. [10] reaches 0.17, 0.39 and 0.51 accuracy on Sine, Stagger and Agrawal streams, whereas the same TFM conditioned on the past rows of the recurring concept reaches at least 0.99; in steady state all contexts are equally accurate.

Observation 2 (Relearning in context is cheap, for simple concepts). A TFM relearns the concepts of the standard drift generators within about 200 rows, so resetting the context

when a drift detector fires [12] already closes most of the gap of Observation 1 (0.81–0.88 in the same five batches). Memory pays only for concepts that take many rows to learn: on mixtures with 100 centroids the TFM needs more than 1000 rows to approach its asymptote.

Observation 3 (Resetting is unsafe). On 29 real streams the same reset detector is on average 1.8 points below the plain FIFO context and up to 9.6 points below it: most rises of the error rate on real streams do not mean that the context has become useless.

These observations raise the question of this paper: when should an in-context learner recall a past concept rather than relearn it, and how can it do so without paying when there is nothing to recall?

Approach. We treat every past context as an offline expert and the current window as an online expert, and combine them with a meta expert, following the mixture of online and offline experts (MOOE) of Zhao et al. [5]. An in-context expert costs no training, which changes both the method and its analysis. (i) Which segments belong to the same concept is decided by a model-oriented discrepancy in the sense of R-divergence [13]; we show that the original form is blind to concepts that contradict each other on the same inputs, which is the defining case of recurring drift, and that an excess-risk form repairs it. (ii) The meta expert has two time scales. A fast level restarts every batch and identifies a recurring concept in one batch under a margin; a slow level decides whether following the fast level is worth it at all, with a worst-case guarantee against the plain window. (iii) The gain of recall over relearning then has a closed form in the TFM’s learning curve, so the value of memory can be measured before deployment.

Contributions. (1) Problem. Recurring concept drift for in-context learners, where past contexts are free experts (Definition 1), with three observations on forgetting, relearning and resetting. (2) Theory. The blind spot of R-divergence under conflicting concepts and its excess-risk repair (Proposition 1); identification in one batch under a margin (Proposition 2); the recall-versus-relearn formula (Theorem 1); and an any-time discounted regret bound for the slow level (Theorem 2). (3) Method. MICE, a training-free mixture of window and stored in-context experts whose every component corresponds to one of these results. (4) Evidence. Pre-registered tests on a controlled grid, where the learning curve predicts the gain of memory and MICE exceeds reset detection, window ensembles, incremental learners and trained MOOE; and 29 real streams, where MICE is the most accurate in-context policy on 23 and stays within 0.25 points of the plain window on all.

2 Recurring Concept Drift for In-Context Learners

Setting. A stream delivers batches of B rows; the labels of a batch arrive one batch late. A frozen TFM $f_{\Theta}(\cdot \mid \mathbf{x}; \mathcal{C})$ predicts every batch from a context $\mathcal{C}$ of at most M labelled rows, and a context policy decides which rows form the context. The stream is a sequence of intervals $I_1, I_2, \dots$ with distributions $P_1, P_2, \dots$ over $(\mathbf{x}, y)$; concept drift occurs between intervals in the sense of Lu et al. [2].

Definition 1 (Recurring in-context concept drift). Let $\mathcal{C}_k$ be a context of rows of interval I_k and $f_{\Theta}(\cdot; \mathcal{C}_k)$ the k -th in-context expert. The drift into interval I_{K+1} is recurring if $\min_{k \leq K} D_T(P_{K+1} \mid P_k) \leq \delta$, where $D_T(Q \mid P)$ is the risk on Q of the expert of P minus that of the expert of Q (Definition 2), and new otherwise.

The definition is model-oriented: a drift is recurring when some stored context serves the new interval as well as its own rows would. Three policies are the reference points of this paper. FIFO is the plain sliding window: the context is the last M labelled rows, first in, first out. It is the simplest context policy and the building block of the short- and long-term memory of Lourenço et al. [10]. Reset empties the context when the drift detection method (DDM) of Gama et al. [12] fires. Window ensemble mixes FIFO contexts of several lengths. None of them has memory.

3 Theory: From a Model-Oriented Discrepancy to Recall versus Relearn

The analysis has four steps, one per component of the method: which contexts share a concept (Section 3.1), how fast a recurring concept is identified (Section 3.2), what recalling it is worth (Section 3.3), and what following the mixture can cost when nothing recurs (Section 3.4). All proofs are in Appendix A.

Notation. For a concept P over $(\mathbf{x}, y)$ and a loss ℓ , the risk of a context $\mathcal{C}$ is

$$\epsilon_P(\mathcal{C}) = \mathbb{E}_{(\mathbf{x}, y) \sim P} \ell(f_{\Theta}(\cdot \mid \mathbf{x}; \mathcal{C}), y),$$

and the learning curve of P is $R_P(n) = \mathbb{E}_{\mathcal{C} \sim P^n} \epsilon_P(\mathcal{C})$. We write $p_{\mathbf{x}} = P(\cdot \mid \mathbf{x})$ for the conditional, P_X for the marginal, H for the entropy and JS for the Jensen–Shannon divergence.

3.1 Which Contexts Share a Concept

R-divergence [13] declares two distributions identical for a model when the hypothesis learned on their mixture has the same risk on both. For an in-context learner that hypothesis is the TFM conditioned on the union of the two contexts:

$$D_R(P \parallel Q) = |\epsilon_P(\mathcal{C}_P \cup \mathcal{C}_Q) - \epsilon_Q(\mathcal{C}_P \cup \mathcal{C}_Q)|. \quad (1)$$

Its population behaviour follows from the principle behind prior-fitted networks, which approximate the posterior predictive distribution [8, 9].

Assumption 1 (Posterior-predictive limit). With a context from P alone, the TFM predicts the conditional $p_{\mathbf{x}}$; with contexts of equal size from P and Q , it predicts the conditional of the mixture $U = \frac{1}{2}(P + Q)$,

$$u_{\mathbf{x}} = \alpha(\mathbf{x}) p_{\mathbf{x}} + (1 - \alpha(\mathbf{x})) q_{\mathbf{x}}, \quad \alpha = \frac{P_X}{P_X + Q_X}.$$

We denote the two predictors by h_P and h_U .

Definition 2 (Recall regret and excess-risk discrepancy). For two concepts P and Q , the recall regret of P on Q is the risk that the context of P incurs on Q beyond that of the context of Q itself, and the excess-risk discrepancy is the mean risk that pooling the two contexts adds to each concept:

$$D_T(Q \mid P) = \epsilon_Q(h_P) - \epsilon_Q(h_Q), \quad (2)$$

$$D_E(P, Q) = \frac{1}{2}[\epsilon_P(h_U) - \epsilon_P(h_P)] + \frac{1}{2}[\epsilon_Q(h_U) - \epsilon_Q(h_Q)]. \quad (3)$$

Proposition 1 (Blind spot of R-divergence and its repair). Let Assumption 1 hold and let ℓ be the log-loss.

- (i) Decomposition. The excess-risk discrepancy is the part of the divergence between the joints that co-variate shift does not explain:

$$D_E(P, Q) = \text{JS}(P, Q) - \text{JS}(P_X, Q_X) \geq 0.$$

If $P_X = Q_X$, then $D_E(P, Q) = \mathbb{E}_{\mathbf{x}} \text{JS}(p_{\mathbf{x}}, q_{\mathbf{x}})$, which is zero if and only if $p_{\mathbf{x}} = q_{\mathbf{x}}$ almost surely.

- (ii) Blind spot. Let $P_X = Q_X$ and let both concepts be deterministic, $y = g_P(\mathbf{x})$ and $y = g_Q(\mathbf{x})$, with disagreement mass $\rho = \Pr_{\mathbf{x}}[g_P(\mathbf{x}) \neq g_Q(\mathbf{x})]$. Then, for every $\rho \in [0, 1]$,

$$D_R(P \parallel Q) = 0 \quad \text{whereas} \quad D_E(P, Q) = \rho \ln 2.$$

- (iii) 0-1 loss. In the setting of (ii) with the 0-1 loss, $D_R(P \parallel Q) = 0$ and $D_T(Q \mid P) = \rho$.

Remark 1. R-divergence asks whether the mixture hypothesis treats the two sets equally. Two concepts that contradict each other on the same inputs are treated equally badly, so the answer is yes although neither context helps the other; this is the defining case of recurring drift. The excess-risk form asks whether pooling costs anything. With finite contexts it has a second term: if $P = Q$, pooling doubles the context and changes the risk by $R_P(2n) - R_P(n) \leq 0$. The empirical excess is interference minus pooling gain, so merging two contexts when it is at most δ merges exactly when pooling helps more than the concepts interfere. MICE uses the directed form (2) as its merge statistic, which is also the regret that recalling the stored expert would incur (Definition 1).

3.2 The Fast Level: Identification under a Margin

Following Zhao et al. [5], the regret of a mixture of experts against the best predictor splits into the regret of the meta expert against the best expert and the regret of that expert,

$$\text{Regret} = \text{Regret}_{\text{ME}} + \text{Regret}_{\text{KE}}. \quad (4)$$

MOOE bounds $\text{Regret}_{\text{ME}} \leq \sqrt{T \ln K}$ for cumulative losses over an interval of T rounds. After a recurrence the cumulative loss still carries the previous concept, so the fast level of MICE restarts every batch: it weights the K experts by

$$w_k \propto \exp(-\eta \ell_k), \quad (5)$$

where ℓ_k is the log-loss of expert k on the last labelled batch only. Its guarantee comes from a margin.

Assumption 2 (Margin). One expert k^* leads the last labelled batch by a margin $g > 0$: $\ell_{k^*} \leq \ell_k - g$ for all $k \neq k^*$.

Lemma 1 (Margin). Under Assumption 2, let $\kappa = \ln(1 + (K - 1)e^{-\eta g})$. The mixture with the weights (5) satisfies:

- (i) the leader holds almost all weight, $w_{k^*} \geq e^{-\kappa}$;
- (ii) on every row, the log-loss of the mixture is at most that of expert k^* plus κ ;
- (iii) the mixture predicts the class of expert k^* on every row where the top two probabilities of that expert differ by more than $e^\kappa - 1$.

Proposition 2 (One batch of identification). Consider a block of J batches of one concept, with log-losses clipped at c , and let Assumption 2 hold for every labelled batch of the block. Then the regret of the fast level per row, summed over the batches of the block, satisfies

$$\text{Regret}_{\text{ME}} \leq c + (J - 1)(K - 1)e^{-\eta g}.$$

Remark 2. The first term is one batch of identification, lost because labels arrive one batch late. The rest is exponentially small in the margin and does not grow with the number of rows, against $\sqrt{JB \ln K}$ for the cumulative rule.

3.3 When Recall Beats Relearning

Let concepts recur in blocks of L rows, that is, $J = L/B$ batches, and let $a_k(n)$ be the accuracy learning curve of concept k , non-decreasing in n . On the c -th visit of concept k ($c \geq 2$) its stored expert holds $n_c = \min(M, (c - 1)L)$ rows. The relearning learner resets its context at the block boundary; in the terms of Zhao et al. [5] it is the online expert, and the stored context is an offline expert that needed no training.

Theorem 1 (Recall versus relearn). Let Assumption 2 hold on every labelled batch of the block for the better of the stored expert and the window, with $e^\kappa - 1$ below the top-two gap of that expert. On the c -th visit of concept k , the expected accuracy of the mixture minus that of relearning, averaged over the block, is

$$\Delta_{k,c}(L) = \frac{1}{J} \sum_{j=1}^{J-1} \left[a_k(n_c) - a_k(\min(jB, M)) \right]_+. \quad (6)$$

Corollary 1 (When memory pays). The gain (6) is zero on the first visit and whenever the learning curve is flat, $a_k(B) = a_k(M)$, and in general

$$\Delta_{k,c}(L) \leq \frac{M}{L} (a_k(M) - a_k(B)).$$

Remark 3. Equation (6) is the area by which the learning curve lies below the accuracy of the stored expert. In the decomposition (4) it is the regret of the online expert minus that of the recalled expert: memory helps when this area exceeds the cost of the meta expert, and Proposition 2 makes that cost one batch, which the relearning learner loses as well. The value of memory is therefore a property of the backbone and the concept that can be measured before deployment, and Corollary 1 explains Observation 2: for concepts that a TFM learns from one batch, the area is zero.

3.4 The Slow Level: A Safeguard at Any Time

Assumption 2 fails on streams whose experts are close to each other. The batch-wise restart then follows noise: the expert that was best on the last batch need not be best on the next, which is the situation of Observation 3. MICE therefore places a second meta expert above the first. It has two experts, the default (the FIFO window of M rows, $k = 0$) and the fast mixture (5) ($k = 1$), with predictions $p_{t,0}$ and $p_{t,1}$ on batch t . Let $\ell_{t,k} \in [0, 1]$ be the mean half Brier score $\frac{1}{2} \|p - e_y\|_2^2$ of expert k on batch t . The slow level keeps discounted losses and predicts with

$$L_{t,k} = \gamma L_{t-1,k} + \ell_{t,k}, \quad L_{0,k} = 0, \quad (7)$$

$$v_{t,k} \propto \exp(-\eta_2 \gamma L_{t-1,k}), \quad (8)$$

$$p_t = v_{t,0} p_{t,0} + v_{t,1} p_{t,1}, \quad (9)$$

with a discount $\gamma \in (0, 1]$. For $\gamma = 1$ this is the cumulative meta expert of MOOE with an exp-concave loss; $\gamma < 1$ forgets losses older than about $1/(1 - \gamma)$ batches.

Theorem 2 (Any-time safeguard). Let K experts be combined by (7)–(9) with a step $\eta_2 \leq \frac{1}{2}$. For every sequence of labels and expert predictions, every time T and every expert k ,

$$\sum_{t=1}^T \gamma^{T-t} (\ell_t(p_t) - \ell_{t,k}) \leq \frac{\ln K}{\eta_2}. \quad (10)$$

Corollary 2 (Cost of following the fast level). With $K = 2$, $\eta_2 = \frac{1}{2}$ and $\gamma = 0.9$, the discounted regret of MICE against the plain window, and against the fast mixture, is at most $2 \ln 2 \approx 1.39$ at every time, against a discounted horizon of $1/(1 - \gamma) = 10$ batches.

Remark 4. Theorem 2 makes no assumption on the stream. The discount makes it an any-time statement: after a change of regime the slow level needs about ten batches, not the length of the stream, to follow the better of the two experts. The two levels divide the work. The fast level identifies a recurring concept within one batch when there is something to identify (Proposition 2); the slow level decides whether identification is worth following. The bounded, exp-concave loss is essential: it confines the step to $\eta_2 \leq \frac{1}{2}$, and the unbounded log-loss lets one confidently wrong batch after a switch dominate the slow level for the whole horizon (Section 5.4).

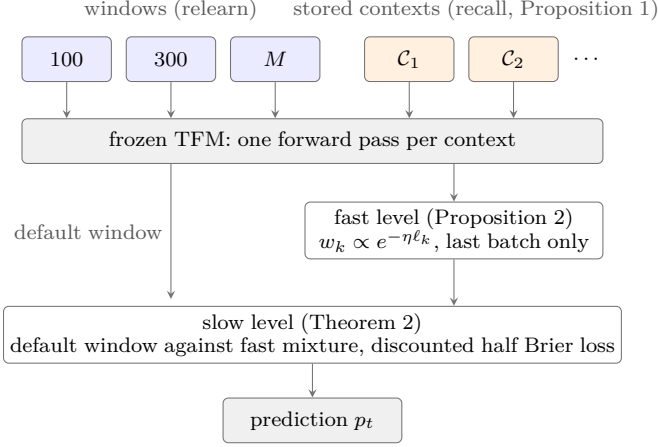


Fig. 1. MICE for one batch. Windows and stored contexts are in-context experts of the same frozen TFM. The fast level mixes all experts from their losses on the last labelled batch; the slow level mixes the result with the default window. Each component is governed by one result of Section 3.

4 Method: A Mixture of In-Context Experts

MICE (Figure 1 and Algorithm 1) runs a frozen TFM; no expert is trained. Each component implements one result of Section 3.

Experts. Windows are FIFO contexts of the last 100, 300 and M labelled rows; they relearn fast and cover concepts that are new or cheap to learn. Stored concepts are contexts kept in a pool. Every 500 labelled rows close a segment S . The segment joins the stored expert $\mathcal{C}_k$ that minimises the estimated recall regret $\widehat{D}_T(S|\mathcal{C}_k) = \widehat{\epsilon}_S(\mathcal{C}_k) - \widehat{\epsilon}_S(S)$ if this is at most $\delta = 0.02$, where $\widehat{\epsilon}_S(S)$ is cross-fitted in two folds because an in-context learner memorises its context; otherwise it finds a new stored expert (Proposition 1). A stored expert keeps its last M rows, and the pool holds at most 12 experts (least recently used out).

Fast level. The windows and the stored experts are mixed with the weights (5), $\eta = 10$ (Lemma 1 and Proposition 2).

Slow level. The fast mixture and the M -row window are mixed by (7)–(9) with $\gamma = 0.9$ and $\eta_2 = \frac{1}{2}$ (Theorem 2 and Corollary 2).

The cost is one TFM call per expert and batch, at most 15, plus three calls when a segment closes. All constants were set on synthetic development streams; none was tuned on a real stream.

5 Experiments

The experiments answer four questions. RQ1: does the learning curve predict the value of memory? RQ2: does MICE recall, that is, exceed relearning and window ensembles when concepts recur? RQ3: is MICE safe and accurate on real streams, where recurrence is weak or absent? RQ4: what does each component contribute, and do the theoretical conditions hold?

Protocol. Prequential evaluation in batches of $B = 100$ rows with labels one batch late; context budget $M = 1000$ for every policy; backbone TabPFN v2 with four ensemble members [6], and TabICL [7] as a second backbone.

Algorithm 1 MICE for one batch t

- 1: Input: unlabelled batch X_t ; labels of batch $t - 1$; windows $\mathcal{W}$; pool $\mathcal{P}$
- 2: update the windows with batch $t - 1$; if 500 new labelled rows: close a segment S and merge it into $\arg \min_{k \in \mathcal{P}} \widehat{D}_T(S|\mathcal{C}_k)$ if the minimum is at most δ , else add S to $\mathcal{P}$
- 3: $\ell_k \leftarrow$ log-loss of expert k on batch $t - 1$, for $k \in \mathcal{W} \cup \mathcal{P}$
- 4: $p^{\text{fast}} \leftarrow \sum_k w_k f_{\Theta}(\cdot | X_t; \mathcal{C}_k)$ with $w_k \propto e^{-\eta \ell_k}$
- 5: $L_k \leftarrow \gamma L_k +$ half Brier score of $k \in \{\text{default, fast}\}$ on batch $t - 1$
- 6: Output: $v_0 f_{\Theta}(\cdot | X_t; \mathcal{C}_{\text{default}}) + v_1 p^{\text{fast}}$ with $v_k \propto e^{-\eta_2 \gamma L_k}$

In-context baselines: FIFO; the dual memory (LTM) of Lourenço et al. [10]; DDM-triggered reset [12]; a window ensemble of the three windows of MICE without the pool. Other stream learners: Adaptive Random Forest [14], Streaming Random Patches [15], the Hoeffding Adaptive Tree [16], Leveraging Bagging [17], ADWIN Bagging, Learn++.NSE [18], and MOOE with trained experts [5], tuned on the development streams over 72 configurations and an extension of its grid (Appendix D).

Streams. A controlled grid of recurring concepts: mixtures of 5, 30 or 100 Gaussian centroids whose class assignment is permuted between three concepts, so that P_X is fixed and concepts conflict; blocks of 500 or 2000 rows; three cycles. Thirty streams from six standard generators (SEA, Stagger, Agrawal, Sine, RBF, Hyperplane) with recurring concepts. Twenty-nine real streams from 13 sources, the long ones in segments of 100,000 rows: Electricity, seven INSECTS streams [19] (one in three segments), Forest CoverType, PokerHand and Airlines (five segments each), Rialto, NOAA Weather, Spam and Phishing.

Pre-registration. Hypotheses, data and decision rules were written down before each test, and every test stream was run once. The real streams were opened in five stages (Table 2). The first three were also used, after their one-time evaluation, to develop the slow level. The third held-out set (eight later segments of four streams) and the grid seeds 20 and 21 were evaluated after the two-level rule was frozen with a step of 10; the step $\frac{1}{2}$ required by Theorem 2 was added to that analysis before any result was computed. The fourth held-out set, six further segments that no method had loaded, was then run once with MICE exactly as defined in Section 4 as the registered method (Appendix D).

5.1 RQ1: The Learning Curve Predicts the Value of Memory

We measured the learning curves of the grid concepts, wrote down the gain predicted by Theorem 1, and only then ran the streams. On seeds 10 and 11 the fast level alone exceeds reset on all eight streams with 30 or 100 centroids, more for shorter blocks and for harder concepts, as predicted, and the rank correlation between predicted and observed gain over the six cells is 0.94; the magnitudes agree as well (0.55 versus 0.53 points, 3.0 versus 2.6, 10.4 versus 8.6). On the fresh seeds 20 and 21 with the full two-level method the correlation is 0.83 (Table 1 and Figure 2). The six standard generators are the other end of the scale: their learning curves are flat after about 200 rows, the predicted gain is at most 0.4 points,

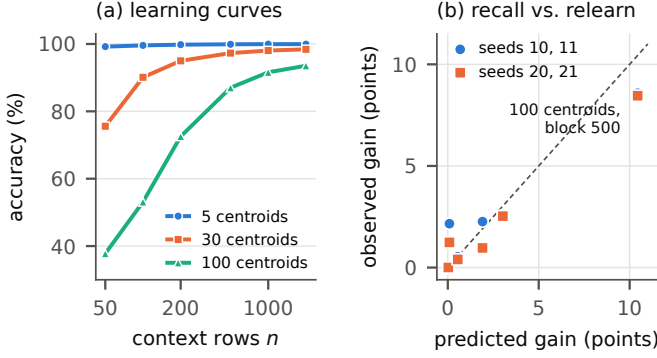


Fig. 2. (a) Learning curves of the grid concepts with TabPFN v2, measured before the grid streams were run. (b) Gain of memory over reset per grid cell: predicted from these curves by Theorem 1 against observed, for the fast level on seeds 10 and 11 and for MICE on seeds 20 and 21; the dashed line is equality.

TABLE 1

Controlled grid, fresh seeds 20 and 21 (accuracy in %, mean of two streams per cell). Win.: window ensemble; Fast: the fast level without the slow level. Pred.: gain over reset predicted by Theorem 1 from learning curves measured beforehand; Gain: observed MICE minus reset, in points.

Cent.	Block	FIFO	Win.	Reset	Fast	MICE	Pred.	Gain
5	500	52.3	82.1	87.3	88.7	88.6	0.1	+1.2
5	2000	85.2	96.3	96.8	96.9	96.8	0.0	+0.0
30	500	40.2	78.8	81.2	82.7	82.2	1.9	+1.0
30	2000	79.3	92.5	93.6	94.3	94.0	0.5	+0.4
100	500	43.0	60.0	63.5	72.4	71.9	10.4	+8.5
100	2000	74.5	81.2	83.2	86.4	85.7	3.0	+2.5
Mean		62.4	81.8	84.3	86.9	86.5		+2.3

and the observed gain of MICE over reset is 0.1 points on average. Recurring-concept memory cannot be evaluated on the standard generators with a TFM; it needs concepts that are costly to learn.

5.2 RQ2: Recall under Recurring Concepts

On the grid MICE exceeds reset on all eight streams with 30 or 100 centroids and the window ensemble on all eight (Table 1): the gain comes from memory, not from faster adaptation. Averaged over the twelve streams it reaches 86.5 against 84.3 for reset, 81.8 for the window ensemble and 62.4 for FIFO; on the hardest cell the gain over reset is 8.5 points. The slow level costs 0.4 points here against the fast level alone, the price of the safeguard. Stream learners that train a model are far behind when a concept must be learned within a block: on seeds 10 and 11, ARF reaches 68.0, SRP 69.7 and the tuned MOOE 78.1, against 83.9 for reset and 86.2 for MICE. On the thirty generator streams MICE matches reset (95.0 against 94.8), and the forgetting of Observation 1 is repaired: 0.82–0.89 in the five batches after a recurrence against 0.17–0.51 for FIFO.

5.3 RQ3: Real Streams

Real streams rarely announce a recurrence, and the experts are close to each other. Here the requirement is safety first. Table 2 shows that every policy that reacts to the last errors

pays for it somewhere: reset loses up to 9.6 points to FIFO (PokerHand) and 1.8 on average, the window ensemble up to 1.4, and the fast level alone up to 2.3. MICE loses at most 0.25 points on any of the 29 streams, is above FIFO on 24 of them (+0.57 on average), and is more accurate than all four in-context baselines on 23. Its largest gains are where the stream revisits its past, Rialto (+4.7 points over FIFO, with a daily cycle) and the INSECTS streams (+0.2 to +1.9).

Confirmatory sets. On the fourth held-out set all four registered hypotheses hold: MICE is not below FIFO or reset on any of the six segments (worst difference to FIFO −0.01 points), it is more accurate than all four in-context baselines on five segments and has the highest mean (85.3 against 85.1 for FIFO, 84.2 for the window ensemble and 80.9 for reset), and it exceeds its own variant without the pool on all six. The fast level alone is 2.0 and 1.9 points below FIFO on the two PokerHand segments, which the slow level turns into +0.4 and +0.5; reset is 9.6 and 8.6 points below. On the third held-out set, MICE is within 0.08 points of FIFO on every segment, has the highest mean (85.2 against 84.9 for FIFO) and is the most accurate policy on five of eight segments; the paired difference to FIFO over batches is +0.30 points (block-bootstrap 95% interval [+0.20, +0.40]), and the registered step of 10 gives the same worst case and five of eight segments.

A second seed. We repeated ten segments with another seed of the backbone, under the same registered hypotheses. On the fourth held-out set all four hold again (most accurate on five of six segments, never below FIFO). On four segments of the third held-out set, non-inferiority, safety and the highest mean hold, and MICE is the most accurate policy on two of the four segments, where the hypothesis asked for three. Over the ten segments the difference to FIFO has the same sign under both seeds and changes by at most 0.4 points; its minimum is −0.08 points.

Beyond accuracy. Table 3 adds the macro-F1 score, the one-vs-rest ROC AUC, the expected calibration error (ECE) and the half Brier score, computed from the stored predictions of every method. On the real streams the picture of the accuracy carries over for F1 and the Brier score: MICE has the highest F1 and a lower Brier score than every baseline; against FIFO its F1 is higher on 24 of 29 streams (+0.8 points on average), its AUC on 25 and its Brier score lower on 26, and against reset on 25, 28 and 28. The exception is the AUC of the dual memory: LTM has the highest mean AUC (91.0 against 90.5), all of it from the five PokerHand segments (up to 7.3 points), where its class-balanced long-term buffer keeps rows of the rare classes that a window loses; on the same segments its accuracy is 2.5 to 3.8 points below MICE, and on the other 24 streams the AUC of MICE is 0.2 points higher in the median. On the grid the step $\eta_2 = \frac{1}{2}$ has a price in the quality of the probabilities: MICE is 0.2 points below the step of 10 in accuracy but 3.1 points in ECE and 1.2 in the Brier score, because a small step keeps weight on the default window, which is poorly calibrated under conflicting concepts (ECE 16.7); its ECE is also above that of reset (7.3), while its Brier score is lower (12.7 against 13.1). The step that Theorem 2 covers is thus the conservative choice: it buys the guarantee with calibration in the recall regime.

Other learners and a second backbone. On all 29 streams, the best trained stream learner per stream (seven learners, MOOE in two configurations) is below MICE on 28 and below

TABLE 2

Real streams: mean accuracy (%) per evaluation stage, and the worst difference to FIFO over the streams of the stage (points). Every stage was run once when it was opened; the third and fourth held-out sets were opened after the two-level rule was frozen. Per-stream results are in Table 4.

Stage	Streams	Mean accuracy						Worst difference to FIFO			
		FIFO	LTM	Reset	Win. ens.	Fast only	MICE	Reset	Win. ens.	Fast only	MICE
development	5	76.6	76.5	76.8	77.1	77.6	77.6	−1.1	−0.7	−0.4	+0.3
first held-out set	4	83.1	82.8	83.3	83.3	83.8	83.8	−0.4	−0.3	−0.2	+0.3
second held-out set	6	84.9	84.3	83.5	84.8	85.1	85.7	−8.1	−1.4	−2.3	−0.2
third held-out set	8	84.9	84.1	82.3	84.4	84.6	85.2	−8.4	−1.2	−1.7	−0.1
fourth held-out set (confirmatory)	6	85.1	83.9	80.9	84.2	84.5	85.3	−9.6	−1.4	−2.0	−0.0
all streams	29	83.3	82.6	81.5	83.0	83.3	83.8	−9.6	−1.4	−2.3	−0.2

TABLE 3

Metrics beyond accuracy (% , mean over streams): macro-F1, one-vs-rest ROC AUC, expected calibration error (15 bins) and half Brier score, on the 29 real streams and the 24 grid streams. Computed from stored predictions; the best value per column is in bold.

Method	Real streams					Controlled grid				
	Acc. ↑	F1 ↑	AUC ↑	ECE ↓	Brier ↓	Acc. ↑	F1 ↑	AUC ↑	ECE ↓	Brier ↓
FIFO	83.3	73.5	89.9	1.7	11.6	61.9	60.9	83.7	16.7	25.1
LTM	82.6	72.1	91.0	1.5	11.9	62.2	61.4	84.8	16.0	24.5
Reset	81.5	71.3	88.5	2.6	12.9	84.1	83.8	91.4	7.3	13.1
Window ensemble	83.0	73.0	89.5	1.2	11.7	81.6	81.0	90.8	8.7	14.9
Fast level only	83.3	73.4	90.3	1.5	11.5	86.8	86.5	93.6	7.1	11.4
MICE, $\eta_2 = 10$	83.8	74.2	90.5	1.1	11.2	86.6	86.3	93.6	7.0	11.5
MICE	83.8	74.3	90.5	1.3	11.2	86.4	86.0	92.3	10.1	12.7

FIFO on 23; the exception is one Airlines segment, where Streaming Random Patches is 0.3 points above MICE. On CoverType and PokerHand the trained learners are 4 to 19 points behind. With TabICL as the backbone and no constant changed, MICE is the most accurate policy on all four streams of the first held-out set (84.0 on average against 83.4 for FIFO and 83.5 for the window ensemble), and in registered tests on four segments of the third held-out set and on all six of the fourth, the four hypotheses of the kind above hold: on the fourth set MICE is the most accurate policy on six of six segments (85.2 on average against 84.7 for FIFO, 84.2 for the window ensemble and 81.3 for reset), and over the ten segments it is at most 0.02 points below FIFO.

5.4 RQ4: Components and Conditions

Windows and pool. Removing the pool, so that the two levels mix the three windows only, lowers accuracy on every one of the 24 grid streams (by 5.6 points on average) and on 28 of the 29 real streams (by 0.35 points on average, 2.1 on Rialto and 1.5 on a PokerHand segment): memory, not the choice of a window, carries the gain, on real streams as well. Removing the two short windows instead lowers the generator streams from 95.0 to 93.6 and the grid from 86.4 to 81.8 and leaves the real streams within 0.2 points on average; the pool forms exactly as many stored experts as a generator stream has concepts.

Slow level. It leaves the recall regime almost unchanged (grid: 86.5 against 86.9 for the fast level alone) and removes the losses on real streams (worst case −0.25 against −2.3; Table 2). With the log-loss in place of the half Brier score, one grid stream falls from 86.4 to 74.9: after a switch the fast mixture is confidently wrong for one batch, and an unbounded loss lets that batch outweigh the next ten.

Discrepancy. On 144 pairs of grid segments, the recall regret and the excess-risk discrepancy separate “different concept” from “same concept” with AUROC 1.00, and the R-divergence estimate with 0.85 (0.42 on the hardest cell), as Proposition 1 predicts; on pairs of the same concept the estimated excess is negative, down to −0.2 for 100 centroids, the pooling gain.

Margin. On the grid the leading expert holds weight 1.000 from the second batch of a recurring block, with a gap of 0.8–8.0 nats to the runner-up, so $(K - 1)e^{-\eta g} < 0.005$ in Proposition 2; on the real streams the median gap is 0.008–0.012 nats, the regime for which the slow level is built.

Regret. Over 33 development streams the largest discounted regret of the slow level is 0.15 against the window and 0.35 against the fast mixture, within the bound of 1.39; on adversarial sequences the bound holds at $\eta_2 = \frac{1}{2}$ and fails for steps of 2 and 10 (Appendix D).

6 Related Work

Concept drift and recurring concepts. Drift detection and adaptation are surveyed by Gama et al. [1] and Lu et al. [2]. Recurring concepts are handled by storing past models and reusing them, selected by statistical tests [3], by meta-learners of each model’s domain [4] or by weighted ensembles of batch learners [18]. MOOE [5] combines offline experts of past intervals with an online expert under a meta expert with regret guarantees; MICE keeps its decomposition and replaces trained experts by stored contexts, the cumulative meta expert by two time scales, and the generic regret by a quantity measured from the learning curve. Model-oriented discrepancy. R-divergence [13] and I-Div [20] measure distribution discrepancy through a learned hypothesis; we identify the blind spot of the former under conflicting concepts and use

an excess-risk form to decide which contexts to pool. TFMs on streams. Lourenço et al. [10] show that a TFM with a short- and long-term FIFO memory outperforms incremental ensembles; Drift-Resilient TabPFN [11] retrains the prior to extrapolate temporal trends. Neither models recurrence. Concept drift has likewise been redefined for other new model families, such as multi-modal large language models [21, 22]. Online aggregation. Exponential weights with exp-concave losses and their logarithmic regret are classical [23, 24]; Theorem 2 is their discounted, any-time form for two in-context experts.

7 Conclusion

For an in-context learner, memory of a past concept is free to store and free to recall, and its value is the area between the learning curve and the accuracy of the stored context. MICE turns this into a method with one component per result: an excess-risk discrepancy decides which contexts share a concept, a fast meta expert identifies a recurring concept in one batch, and a slow meta expert guarantees, at any time and for any stream, that following it costs little against the plain window. When concepts recur and are costly to learn, MICE exceeds reset detection, window ensembles and trained experts by a wide margin; on 29 real streams it is the most accurate in-context policy on 23 and never more than 0.25 points below a sliding window, where reset detection loses up to 9.6 points.

Limitations. The gains on real streams other than Rialto are below two points; the claim there is safety with a small average gain, and the large gains are for recurring concepts with steep learning curves. The confirmatory real segments come from three streams whose earlier segments were used in development; ten segments were repeated with a second seed and the others have one run. Theorem 2 bounds the half Brier score rather than the accuracy, its step costs calibration when concepts recur (Table 3), and Proposition 1 holds under Assumption 1, and Theorem 1 under Assumption 2. MICE calls the TFM once per expert, 5–18 times the time of a single context in our runs.

References

- [1] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, “A survey on concept drift adaptation,” *ACM Comput. Surv.*, vol. 46, no. 4, pp. 44:1–44:37, 2014.
- [2] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, “Learning under concept drift: A review,” *IEEE Trans. Knowl. Data Eng.*, vol. 31, no. 12, pp. 2346–2363, 2019.
- [3] P. M. Gonçalves, Jr. and R. S. M. de Barros, “RCD: A recurring concept drift framework,” *Pattern Recognit. Lett.*, vol. 34, pp. 1018–1025, 2013.
- [4] J. Gama and P. Kosina, “Recurrent concepts in data streams classification,” *Knowl. Inf. Syst.*, vol. 40, 2014.
- [5] Z. Zhao, L. Cao, and Y. Wan, “Mixture of online and offline experts for non-stationary time series,” in *Proc. AAAI*, 2025.
- [6] N. Hollmann, S. Müller, L. Purucker, A. Krishnakumar, M. Körfer, S. B. Hoo, R. T. Schirmer, and F. Hutter, “Accurate predictions on small data with a tabular foundation model,” *Nature*, vol. 637, no. 8045, pp. 319–326, 2025.
- [7] J. Qu, D. Holzmüller, G. Varoquaux, and M. L. Morvan, “TabICL: A tabular foundation model for in-context learning on large data,” in *Proc. ICML*, 2025, pp. 50 817–50 847.
- [8] S. Müller, N. Hollmann, S. Pineda-Arango, J. Grabocka, and F. Hutter, “Transformers can do Bayesian inference,” in *Proc. ICLR*, 2022.
- [9] T. Nagler, “Statistical foundations of prior-data fitted networks,” in *Proc. ICML*, 2023, pp. 25 660–25 676.
- [10] A. Lourenço, J. Gama, E. P. Xing, and G. Marreiros, “In-context learning of evolving data streams with tabular foundational models,” in *Proc. ACM SIGKDD*, 2026.
- [11] K. Helli, D. Schnurr, N. Hollmann, S. Müller, and F. Hutter, “Drift-resilient TabPFN: In-context learning temporal distribution shifts on tabular data,” in *Proc. NeurIPS*, 2024, pp. 98 742–98 781.
- [12] J. Gama, P. Medas, G. Castillo, and P. Rodrigues, “Learning with drift detection,” in *Advances in Artificial Intelligence – SBIA 2004*, ser. Lecture Notes in Computer Science, vol. 3171. Springer, 2004, pp. 286–295.
- [13] Z. Zhao and L. Cao, “R-divergence for estimating model-oriented distribution discrepancy,” in *Proc. NeurIPS*, 2023.
- [14] H. M. Gomes, A. Bifet, J. Read, J. P. Barddal, F. Enembreck, B. Pfahringer, G. Holmes, and T. Abdesslem, “Adaptive random forests for evolving data stream classification,” *Mach. Learn.*, vol. 106, no. 9–10, pp. 1469–1495, 2017.
- [15] H. M. Gomes, J. Read, and A. Bifet, “Streaming random patches for evolving data stream classification,” in *Proc. IEEE International Conference on Data Mining (ICDM)*, 2019, pp. 240–249.
- [16] A. Bifet and R. Gavaldà, “Adaptive learning from evolving data streams,” in *Proc. International Symposium on Intelligent Data Analysis (IDA)*, 2009, pp. 249–260.
- [17] A. Bifet, G. Holmes, and B. Pfahringer, “Leveraging bagging for evolving data streams,” in *Proc. ECML PKDD*, 2010, pp. 135–150.
- [18] R. Elwell and R. Polikar, “Incremental learning of concept drift in nonstationary environments,” *IEEE Trans. Neural Networks*, vol. 22, no. 10, pp. 1517–1531, 2011.
- [19] V. M. A. Souza, D. M. dos Reis, A. G. Maletzke, and G. E. A. P. A. Batista, “Challenges in benchmarking stream learning algorithms with real-world data,” *Data Mining and Knowledge Discovery*, vol. 34, no. 6, pp. 1805–1858, 2020.
- [20] Z. Zhao, L. Cao, X. Fan, and W.-S. Zheng, “Revealing distribution discrepancy by sampling transfer in unlabeled data,” in *Proc. NeurIPS*, 2024.
- [21] X. Yang, J. Lu, and E. Yu, “Adapting multi-modal large language model to concept drift from pre-training onwards,” in *Proc. ICLR*, 2025.
- [22] X. Yang, E. Yu, W. Duan, and J. Lu, “Turning drift into constraint: Robust reasoning alignment in non-stationary multi-stream environments,” in *Proc. ICML*, 2026.
- [23] N. Cesa-Bianchi and G. Lugosi, *Prediction, Learning, and Games*. Cambridge University Press, 2006.
- [24] E. Hazan, A. Agarwal, and S. Kale, “Logarithmic regret algorithms for online convex optimization,” *Mach. Learn.*,

vol. 69, pp. 169–192, 2007.

- [25] J. Montiel, M. Halford, S. M. Mastelini, G. Bolmier, R. Sourty, R. Vaysse, A. Zouitine, H. M. Gomes, J. Read, T. Abdessalem, and A. Bifet, “River: Machine learning for streaming data in Python,” *J. Mach. Learn. Res.*, vol. 22, no. 110, 2021.

Appendix

.1 Proof of Proposition 1

Proof. For the log-loss and any conditional h ,

$$\epsilon_P(h) = \mathbb{E}_{P_X} [H(p_{\mathbf{x}}) + \text{KL}(p_{\mathbf{x}} \| h_{\mathbf{x}})],$$

so that $\epsilon_P(h_P) = \mathbb{E}_{P_X} H(p_{\mathbf{x}})$ and $\epsilon_P(h_U) - \epsilon_P(h_P) = \mathbb{E}_{P_X} \text{KL}(p_{\mathbf{x}} \| u_{\mathbf{x}})$, and likewise for Q .

(i) By the chain rule, $\text{KL}(P \| U) = \text{KL}(P_X \| U_X) + \mathbb{E}_{P_X} \text{KL}(p_{\mathbf{x}} \| u_{\mathbf{x}})$, and the same holds for Q . Averaging the two identities gives

$$\text{JS}(P, Q) = \text{JS}(P_X, Q_X) + D_E(P, Q),$$

and $D_E \geq 0$ as a mean of KL terms. If $P_X = Q_X$, then $\alpha \equiv \frac{1}{2}$, $u_{\mathbf{x}} = \frac{1}{2}(p_{\mathbf{x}} + q_{\mathbf{x}})$ and $D_E = \mathbb{E}_{\mathbf{x}} \text{JS}(p_{\mathbf{x}}, q_{\mathbf{x}})$, which vanishes if and only if $p_{\mathbf{x}} = q_{\mathbf{x}}$ almost surely.

(ii) By the display above,

$$\begin{aligned} D_R(P \| Q) &= \left| \mathbb{E}_{P_X} [H(p_{\mathbf{x}}) + \text{KL}(p_{\mathbf{x}} \| u_{\mathbf{x}})] \right. \\ &\quad \left. - \mathbb{E}_{Q_X} [H(q_{\mathbf{x}}) + \text{KL}(q_{\mathbf{x}} \| u_{\mathbf{x}})] \right|. \end{aligned}$$

Deterministic concepts have $H(p_{\mathbf{x}}) = H(q_{\mathbf{x}}) = 0$, and $u_{\mathbf{x}}$ puts mass $\frac{1}{2}$ on each of $g_P(\mathbf{x})$ and $g_Q(\mathbf{x})$ where they differ and mass 1 where they agree. Hence $\text{KL}(p_{\mathbf{x}} \| u_{\mathbf{x}}) = \text{KL}(q_{\mathbf{x}} \| u_{\mathbf{x}}) = \ln 2 \cdot \mathbf{1}[g_P(\mathbf{x}) \neq g_Q(\mathbf{x})]$; with $P_X = Q_X$ the two expectations cancel, and $D_E = \rho \ln 2$.

(iii) Under the 0-1 loss, with ties of $u_{\mathbf{x}}$ broken independently of the source of the query, the prediction of h_U at a disagreeing $\mathbf{x}$ is $g_P(\mathbf{x})$ or $g_Q(\mathbf{x})$ with probability $\frac{1}{2}$ each. Each risk is therefore $\rho/2$ and $D_R = 0$. The predictor h_P errs on Q exactly where the concepts disagree and h_Q nowhere, so $D_T(Q | P) = \rho$. $\square$

.2 Proofs of Lemma 1 and Proposition 2

Proof of Lemma 1. (i) By Assumption 2,

$$\begin{aligned} w_{k^*} &= \left(1 + \sum_{k \neq k^*} e^{-\eta(\ell_k - \ell_{k^*})} \right)^{-1} \\ &\geq (1 + (K-1)e^{-\eta g})^{-1} = e^{-\kappa}. \end{aligned}$$

(ii) For a row with label y and expert predictions p_k ,

$$-\ln \sum_k w_k p_k(y) \leq -\ln(w_{k^*} p_{k^*}(y)) \leq -\ln p_{k^*}(y) + \kappa.$$

(iii) For two classes y_1, y_2 ,

$$\begin{aligned} \sum_k w_k (p_k(y_1) - p_k(y_2)) &\geq w_{k^*} (p_{k^*}(y_1) - p_{k^*}(y_2)) \\ &\quad - (1 - w_{k^*}), \end{aligned}$$

which is positive when the gap of expert k^* exceeds $(1 - w_{k^*})/w_{k^*} \leq e^{\kappa} - 1$. $\square$

Proof of Proposition 2. The first batch of the block is predicted with weights from a batch of the previous concept; its per-row regret is at most c . Every later batch is predicted with weights from a labelled batch of the block, on which k^* leads by g , so Lemma 1(ii) bounds its per-row regret by $\ln(1 + (K-1)e^{-\eta g}) \leq (K-1)e^{-\eta g}$. Summing over the $J-1$ later batches gives the claim. $\square$

.3 Proofs of Theorem 1 and Corollary 1

Proof of Theorem 1. Both learners lose the first batch of the block, whose labels have not arrived. Before batch $j + 1$ ($j = 1, \dots, J - 1$) the relearning context holds $\min(jB, M)$ rows of the block, so its expected accuracy is $a_k(\min(jB, M))$; the stored expert has expected accuracy $a_k(n_c)$. By Lemma 1(iii) the mixture predicts as the better of the two, so its expected accuracy is the larger of the two values, and the difference to relearning is the positive part in (6). Averaging over the J batches gives the claim. $\square$

Proof of Corollary 1. On the first visit the pool holds no expert of the concept, so the mixture and the relearning learner use the same rows. Since a_k is non-decreasing and $n_c \leq M$, every term of (6) is at most $a_k(M) - a_k(B)$, which is zero for a flat curve, and the terms with $jB \geq M$ vanish. At most M/B of the $J = L/B$ terms are non-zero, which gives the bound. $\square$

.4 Proofs of Theorem 2 and Corollary 2

Proof of Theorem 2. Exp-concavity. The half Brier score $\ell(p) = \frac{1}{2}\|p - e_y\|^2$ has gradient $p - e_y$ and Hessian I , and $\|p - e_y\|^2 \leq 2$ on the simplex. Hence $\nabla^2 \ell \succeq \eta_2 \nabla \ell \nabla \ell^\top$ for $\eta_2 \leq \frac{1}{2}$, that is, ℓ is η_2 -exp-concave. The mean over the rows of a batch is η_2 -exp-concave as well: its Hessian is the mean Hessian, and $\frac{1}{n} \sum_i g_i g_i^\top \succeq \bar{g} \bar{g}^\top$ for the row gradients g_i with mean $\bar{g}$. Consequently

$$e^{-\eta_2 \ell_t(p_t)} \geq \sum_k v_{t,k} e^{-\eta_2 \ell_{t,k}}. \quad (11)$$

Potential. Let $W_t = \sum_k e^{-\eta_2 L_{t,k}}$, so that $W_0 = K$. By (7) and (8),

$$W_t = \left(\sum_k e^{-\eta_2 \gamma L_{t-1,k}} \right) \sum_k v_{t,k} e^{-\eta_2 \ell_{t,k}}.$$

By the power-mean inequality, $\sum_k x_k^\gamma \leq K^{1-\gamma} (\sum_k x_k)^\gamma$ for $x_k \geq 0$ and $\gamma \in (0, 1]$; with $x_k = e^{-\eta_2 L_{t-1,k}}$ and (11),

$$\ln W_t \leq (1 - \gamma) \ln K + \gamma \ln W_{t-1} - \eta_2 \ell_t(p_t).$$

Unrolling. Since $(1 - \gamma) \sum_{t \leq T} \gamma^{T-t} + \gamma^T = 1$,

$$\ln W_T \leq \ln K - \eta_2 \sum_{t=1}^T \gamma^{T-t} \ell_t(p_t).$$

On the other hand $\ln W_T \geq -\eta_2 L_{T,k} = -\eta_2 \sum_{t=1}^T \gamma^{T-t} \ell_{t,k}$ for every k . Combining the two inequalities gives (10). $\square$

Proof of Corollary 2. Insert $K = 2$ and $\eta_2 = \frac{1}{2}$ into (10); the discounted number of batches is $\sum_{t \leq T} \gamma^{T-t} \leq 1/(1 - \gamma)$. $\square$

All hypotheses below were written to the experiment log before the corresponding streams were run, and every test stream was run once; interrupted runs were resumed from their checkpoints without rerunning a finished cell.

Grid, seeds 10 and 11 (fast level alone). Hypotheses: gain over reset positive on all eight streams with 30 or 100 centroids, larger for shorter blocks and for more centroids; Spearman correlation between predicted and observed gain at least 0.8; the window ensemble below the method on these eight streams. All held (correlation 0.94). An incidental expectation of no gain with 5 centroids was missed on one stream, where the reset baseline detected late.

Generator streams, seeds 10–14, and five real development streams. Non-inferiority to reset on the generator streams (+0.50 points, 25 of 30 positive) and on each real stream held, as did the replication of Observation 1. An exploratory expectation that the INSECTS stream with re-occurring drift would gain most was not met (it ranked second).

First held-out set (four streams; fast level alone). Non-inferiority to reset on every stream and highest mean accuracy held; the method was the most accurate on three of four streams. A replication with TabICL held as well.

Second held-out set (six streams; fast level alone). Non-inferiority to reset held. The hypothesis that the fast level alone would be the most accurate method on at least four of six streams was not met (one of six), and on PokerHand it was 2.3 points below FIFO. This result motivated the slow level; all real streams seen so far count as development data from here on.

Third held-out set (eight segments; two-level rule frozen). Two candidates for the slow level were registered before any result existed: the log-loss with step 1, and the half Brier score with step 10, added after the log-loss failed on a cached synthetic test stream. For both, non-inferiority to FIFO and reset (tolerance 0.5 points) and to the fast level alone held on every segment, and the mean was the highest. “Most accurate on at least five of eight segments” held for the half Brier score (five) and not for the log-loss (four). The step $\eta_2 = \frac{1}{2}$ with tempered weights, the method of this paper, was added to the analysis after the baselines of three segments were visible and before any result of a two-level rule was computed; it was not part of the registered hypotheses. It gives five of eight segments and the highest mean.

Fourth held-out set (six segments: the next 200,000 rows each of Airlines, PokerHand and CoverType; method of Section 4 frozen). Registered hypotheses: on every segment not more than 0.5 points below the better of FIFO and reset; above all four in-context baselines on at least four of six segments, with the highest mean; on every segment not more than 0.3 points below FIFO; above the variant without the pool on at least four of six segments. All held (five of six segments; worst difference to FIFO -0.01 points; six of six against the variant without the pool).

Second backbone and second seed. With the method of Section 4 frozen and the same four hypotheses (“at least three of four” for sets of four segments): TabICL on four segments of the third held-out set and on the six segments of the fourth, all held; a second seed of TabPFN on the same four segments, three held and “most accurate on at least three of four segments” did not (two); a second seed on the six segments of the fourth held-out set, all held.

Grid, seeds 20 and 21 (half Brier score, step 10). Mean difference to the fast level alone at least -0.5 points (-0.14), above reset on all eight streams with 30 or 100 centroids, and within one point of reset with 5 centroids: all held. The step $\frac{1}{2}$ is 0.23 points lower on average and satisfies the same three statements.

Streams. The grid and generator streams are produced by `code/grid_streams.py` and `code/streams.py`; generator streams have blocks of 2000 rows and three cycles over three or four concepts. Real streams with more than 100,000 rows are cut into segments of 100,000 (CoverType and one INSECTS stream: 150,000 for the first segment). The second

held-out set consists of all streams of a public collection of drift data sets that have at least 5000 rows, at most ten classes and had not been used before. Baselines. LTM follows Algorithm 1 of Lourenço et al. [10] with a short-term share of 0.75. MOOE follows the original algorithm with linear experts on random Fourier features, generalised to several classes; it was tuned on the five development streams over feature type, interval length, step size and regularisation (72 configurations), and its grid was extended twice because the optimum lay on the boundary; the best configuration of every round is evaluated. The incremental learners use the default settings of river [25] with ten base models and the same batch protocol. Regret under adversarial sequences. With synthetic expert predictions and labels chosen to hurt the mixture or the current leader ($K \in \{2, 5\}$, $\gamma \in \{0.5, 0.9, 0.99, 1\}$, 400 rounds), the discounted regret at $\eta_2 = \frac{1}{2}$ stays below $\ln K/\eta_2$ in every configuration (at most 0.67 for $K = 2$), exceeds it at $\eta_2 = 2$ for $\gamma \geq 0.9$, and reaches 4.4 at $\eta_2 = 10$, $\gamma = 0.9$.

TABLE 4

Real streams: accuracy (%) per stream. Fast only: the fast level without the slow level; $\eta_2 = 10$: the pre-registered step; Trained: the best trained stream learner per stream; Δ : MICE minus FIFO in points.

Stream	FIFO	LTM	Reset	Win. ens.	Fast only	$\eta_2 = 10$	MICE	Trained	Δ
development									
elec2	81.0	80.8	79.9	80.7	80.6	81.2	81.2	80.1	+0.27
ins-abrupt-bal	76.5	76.3	76.5	77.4	78.0	77.9	77.9	76.7	+1.32
ins-gradual-bal	78.8	79.2	79.9	79.9	80.7	80.6	80.4	79.3	+1.59
ins-incr-bal	67.5	67.4	67.5	66.8	67.8	67.8	67.8	65.5	+0.30
ins-incr-reoc-bal	79.1	78.7	80.1	80.6	81.0	80.9	80.8	79.2	+1.65
mean	76.6	76.5	76.8	77.1	77.6	77.7	77.6	76.1	+1.03
first held-out set									
covertime	94.8	94.7	94.3	94.9	94.5	95.0	95.1	91.9	+0.27
ins-abrupt-imb	79.4	79.0	79.6	79.2	79.8	79.8	79.8	79.3	+0.41
ins-gradual-imb	81.6	81.3	81.7	81.4	81.9	81.9	81.9	81.4	+0.25
ins-incr-abrupt-bal	76.7	76.2	77.7	77.9	78.8	78.8	78.6	76.9	+1.86
mean	83.1	82.8	83.3	83.3	83.8	83.9	83.8	82.4	+0.70
second held-out set									
airlines	69.0	68.5	69.0	68.1	68.8	69.0	69.1	68.2	+0.05
phishing	94.6	94.6	94.6	94.1	94.6	94.6	94.6	94.4	+0.03
poker	92.6	89.9	84.4	91.1	90.2	92.5	93.0	82.8	+0.41
rialto	77.5	77.5	77.9	80.8	82.0	82.2	82.2	64.3	+4.73
spam	94.5	94.7	94.1	94.2	94.2	94.5	94.2	91.5	-0.25
weather	81.1	80.5	81.1	80.2	80.7	81.1	81.1	79.3	-0.01
mean	84.9	84.3	83.5	84.8	85.1	85.7	85.7	80.1	+0.83
third held-out set									
airlines-b	70.6	70.0	70.5	69.9	70.2	70.5	70.5	70.0	-0.08
airlines-c	65.1	64.5	65.0	64.3	65.0	65.2	65.2	65.0	+0.11
covertime-b	96.6	96.6	95.7	96.6	96.4	96.6	96.6	91.9	-0.03
covertime-c	95.7	95.8	91.9	95.7	95.7	95.7	95.7	80.2	+0.02
ins-b	81.4	81.0	81.4	81.2	81.7	81.7	81.7	81.1	+0.25
ins-c	85.4	85.0	85.5	85.3	85.7	85.7	85.7	85.5	+0.23
poker-b	93.4	91.5	85.0	92.2	91.7	93.7	94.0	76.5	+0.58
poker-c	90.9	88.3	83.4	89.8	90.2	91.7	92.2	74.6	+1.29
mean	84.9	84.1	82.3	84.4	84.6	85.1	85.2	78.1	+0.30
fourth held-out set (confirmatory)									
airlines-d	65.8	64.7	65.8	64.5	65.9	66.0	66.0	66.3	+0.18
airlines-e	65.5	64.8	65.3	64.2	65.5	65.6	65.6	65.0	+0.14
covertime-d	96.2	96.2	89.7	96.2	96.2	96.2	96.2	77.6	+0.05
covertime-e	97.2	97.2	96.7	97.2	97.1	97.2	97.2	92.9	-0.00
poker-d	94.1	91.6	84.6	92.9	92.1	94.1	94.5	77.9	+0.35
poker-e	91.8	89.1	83.2	90.5	89.9	92.4	92.3	75.8	+0.51
mean	85.1	83.9	80.9	84.2	84.5	85.2	85.3	75.9	+0.20
All streams	83.3	82.6	81.5	83.0	83.3	83.8	83.8		+0.57